Chapter 3 Conditionals and Loops

CHAPTER OBJECTIVES

- Discuss the flow of control through a method.
- Explore boolean expressions that can be used to make decisions.
- Perform basic decision making using `if` and `switch` statements.
- Discuss issues pertaining to the comparison of certain types of data.
- Execute statements repetitively using `while`, `do`, and `for` loops.
- Discuss the concept of an iterator object and use one to read a text file.

Every programming language provides mechanisms for making decisions about what to do next. Some of those mechanisms also allow a particular action to be carried out repeatedly. This chapter examines several such constructs, while also exploring subtleties involved in comparing different kinds of data and objects. We begin with boolean expressions, which serve as the foundation for any decision-making logic.

3.1 Boolean Expressions

The sequence in which statements are carried out during program execution is known as the flow of control. In the absence of any special constructs, execution proceeds linearly — the program starts at the first statement and advances through each subsequent statement in order until it reaches the end. In a Java application, this means execution begins at the first line of the `main` method and moves step by step to the final line.

Calling a method diverts the flow of control. When a method is invoked, execution jumps to the body of that method and runs through its statements. When the method finishes, control returns to the point in the calling code immediately following the invocation and continues from there. Method invocation and its mechanics are examined more closely in the next chapter.

Within any given method, the flow of control can be deliberately shaped using specialized programming constructs. These constructs fall into two broad families: conditionals and loops.

KEY CONCEPT

Conditionals and loops allow us to control the flow of execution through a method.

A conditional statement is sometimes referred to as a selection statement because it enables the programmer to choose which statement will be executed next. Java provides three conditional constructs: the `if` statement, the `if-else` statement, and the `switch` statement. Each of these allows the program to determine at runtime which path of execution to follow.

Every such decision is rooted in a boolean expression — also called a condition — which is an expression that evaluates to either `true` or `false`. The outcome of that evaluation determines which statement is executed next.

KEY CONCEPT
An `if` statement allows a program to choose whether to execute a particular statement.

The following is a representative example of an `if` statement:

```
if (count > 20)
   System.out.println ("Count exceeded");
```

The condition here is `count > 20`. This expression produces a boolean result: either the value in `count` exceeds `20` or it does not. If the condition holds, the `println` statement executes. If it does not, the `println` statement is bypassed and execution continues with whatever follows. Conditional statements are covered in thorough detail throughout this chapter.

The need to make decisions of this kind arises constantly in real programming scenarios. Consider life insurance pricing: the premium calculation differs depending on whether the policyholder is a smoker. A conditional statement evaluates that boolean condition and then directs execution to the appropriate calculation.

KEY CONCEPT
A loop allows a program to execute a statement multiple times.

A loop — also called a repetition statement — allows a statement or group of statements to be executed again and again. Like a conditional, a loop is governed by a boolean expression that determines how many times the body of the loop is executed.

As a practical illustration, suppose we need to compute the grade point average for every student in a class. The calculation itself is identical for each student; only the data changes. A loop allows us to repeat that calculation for each student in turn, continuing until all students have been processed.

Java provides three distinct loop constructs: the `while` statement, the `do` statement, and the `for` statement. Each has its own structural characteristics that make it well suited to particular situations.

All conditionals and loops are grounded in boolean expressions, which are built using equality operators, relational operators, and logical operators. Before diving into the specifics of each conditional and loop construct, let's take a closer look at these operators.

Equality and Relational Operators

The `==` and `!=` operators are known as the equality operators. They test whether two values are equal or unequal, respectively. It is important not to confuse the equality operator (`==`), which consists of two consecutive equal signs, with the assignment operator (`=`), which uses only one.

The following `if` statement prints a line of output only when the variables `total` and `sum` hold identical values:

```
if (total == sum)
   System.out.println ("total equals sum");
```

Conversely, the following statement prints output only when `total` and `sum` differ:

```
if (total != sum)
   System.out.println ("total does NOT equal sum");
```

Java also provides a set of relational operators for determining the relative ordering of two values. We already saw the greater-than operator (`>`) in an earlier example. The full set of relational operators covers the other directional comparisons as well: less than (`<`), greater than or equal to (`>=`), and less than or equal to (`<=`). Figure 4.1 presents all of the equality and relational operators Java supports.

Both the equality and relational operators rank lower in precedence than the arithmetic operators. This means arithmetic operations are fully resolved before any equality or relational comparisons take place. As always, parentheses can be employed to override the default evaluation order whenever needed.

Additional examples of relational operators will appear throughout the remainder of this chapter as we work through each conditional and loop construct.

Logical Operators

Beyond equality and relational operators, Java defines three logical operators that accept boolean operands and produce boolean results. These are described in Figure 4.2.

The `!` operator performs logical NOT, also referred to as the logical complement. It is a unary operator — it operates on a single boolean operand — and it returns the opposite of that operand's value. If a boolean variable named `found` is `false`, then `!found` evaluates to `true`. If `found` is `true`, then `!found` is `false`. Importantly, the `!` operator does not modify the stored value of `found`; it simply produces an expression with the opposite boolean value.

The behavior of any logical operator can be described systematically using a truth table, which enumerates every possible combination of input values and the corresponding output.

Since `!` is unary, its truth table has only two rows — one for each possible value of its single operand. Figure 4.3 shows this truth table.

The `&&` operator performs logical AND. Its result is `true` only when both operands are `true`; in all other cases the result is `false`. The `||` operator performs logical OR. Its result is `true` when at least one of its operands is `true`, and `false` only when both operands are `false`.

Both `&&` and `||` are binary operators, each taking two operands. With two operands, there are four possible input combinations to consider. Figure 4.4 presents the truth table covering both operators across all four cases.

Among the three logical operators, `!` has the highest precedence, followed by `&&`, with `||` having the lowest.

Logical operators enable the construction of sophisticated conditions for decision making. Consider the following example:

```
if (!done && (count > MAX))
   System.out.println ("Completed.");
```

The `println` statement executes only when two things are both true simultaneously: `done` is `false` (so `!done` is `true`), and `count` exceeds `MAX`. Figure 4.5 provides a truth table that traces all possible input combinations for this specific condition.

KEY CONCEPT
Logical operators can be used to construct sophisticated conditions.

An important behavioral characteristic of `&&` and `||` in Java is that both are short-circuit evaluated. This means that if the left-hand operand alone is sufficient to determine the overall result, the right-hand operand is never evaluated. For `&&`, if the left operand is `false`, the result must be `false` regardless of the right operand — so the right side is skipped. For `||`, if the left operand is `true`, the result must be `true` regardless of the right operand — so again the right side is skipped.

Short-circuit evaluation can sometimes be deliberately leveraged. The following condition, for example, will never attempt a division by zero if `count` is zero, because the left side of `&&` will evaluate to `false` first, preventing the right side from being evaluated at all:

```
if (count != 0 && total/count > MAX)
   System.out.println ("Testing.");
```

That said, relying on short-circuit behavior requires careful consideration. Not all programming languages guarantee this behavior, and code that depends on it may produce division-by-zero errors when ported to such environments. As always, clarity and readability

should guide these decisions — the logic of your program should be transparent to anyone reading it.

3.2 The `if` Statement

We have already encountered the `if` statement in earlier examples. Let's now examine it systematically.

An `if` statement consists of the keyword `if`, followed by a boolean condition enclosed in parentheses, followed by a statement to execute if the condition is true. If the condition evaluates to `true`, that statement is executed and control then proceeds to whatever follows the `if` statement. If the condition evaluates to `false`, the governed statement is bypassed entirely and execution jumps immediately to whatever code comes after. Figure 4.6 illustrates this decision logic visually.

Consider the following example:

```
if (total > amount)

   total = total + (amount + 1);
```

If `total` is greater than `amount`, the assignment executes; otherwise it is skipped entirely.

KEY CONCEPT
Proper indentation is important for human readability; it shows the relationship between one statement and another.

Notice that the assignment statement is indented beneath the `if` header line. This visual formatting communicates to any human reader that the assignment is governed by the `if` — that its execution is conditional. Indentation carries no meaning to the compiler, but it is critically important for human comprehension.

The `Age` program in Listing 3.1 reads the user's age and conditionally prints a message depending on the value entered.

In every case, the program echoes the entered age back to the screen. If the age falls below the value of the constant `MINOR`, the message about youth is displayed. If the age equals or exceeds `MINOR`, that `println` is skipped. The final sentence about age being a state of mind is printed unconditionally in all cases.

A few additional `if` examples are worth examining. The following sets `size` to zero whenever its current value equals or exceeds `MAX`:

```
if (size >= MAX)

   size = 0;
```

The condition in the next example adds three values before making its comparison:

```
if (numBooks < stackCount + inventoryCount + duplicateCount)
    reorder = true;
```

If `numBooks` is less than the combined total of the other three variables, `reorder` is set to `true`. The addition operations are resolved before the less-than comparison because arithmetic operators have higher precedence than relational operators.

The following example uses the return value from a `Random` object to determine a winner:

```
if (generator.nextInt(CHANCE) == 0)
    System.out.println ("You are a randomly selected winner!");
```

The probability of winning is governed by the value of `CHANCE`. If `CHANCE` is `20`, the odds are 1 in 20. The choice to check for a return value of `0` is arbitrary — any value from `0` to `CHANCE - 1` would serve the same purpose.

The `if-else` Statement

There are many situations where we want one action to occur when a condition is true and a different action when it is false. Adding an `else` clause to an `if` statement creates an `if-else` statement that handles both cases explicitly:

```
if (height <= MAX)
    adjustment = 0;
else
    adjustment = MAX - height;
```

KEY CONCEPT
An `if-else` statement allows a program to do one thing if a condition is true and another thing if the condition is false.

When the condition is true, the first assignment executes. When the condition is false, the second assignment executes. Because a boolean expression can only be `true` or `false`, exactly one of the two branches will always execute — never both, never neither. Proper indentation is again used to make clear that both assignment statements are governed by the enclosing `if-else`.

The `Wages` program in Listing 3.2 demonstrates an `if-else` statement used to calculate an employee's pay, accounting for overtime hours when applicable.

When the hours worked exceed the standard 40-hour threshold, overtime hours are compensated at one and a half times the regular rate. When hours do not exceed the threshold, the total pay is simply hours multiplied by the standard rate.

Consider the following example involving an object:

```
if (roster.getSize() == FULL)
   roster.expand();
else
   roster.addName (name);
```

Even without knowing what `roster` represents, we can read this clearly: if the roster has reached capacity, it is expanded; otherwise, a new name is added to it. The object exposes at least three methods — `getSize`, `expand`, and `addName` — and the `if-else` statement selects which of the latter two to invoke based on the result of calling the first.

Using Block Statements

When the outcome of a boolean condition should trigger more than one statement, Java allows any single statement in a conditional to be replaced with a block statement — a group of statements enclosed in curly braces. We have already used braces extensively to delimit method and class bodies; here they serve the same grouping function within conditional logic.

The `Guessing` program in Listing 3.3 demonstrates an `if-else` where the `else` branch contains a block statement.

If the user's guess matches the randomly chosen answer, a congratulatory message is printed. If the guess is wrong, two messages are printed — one confirming the guess was incorrect, and one revealing the actual answer. Both of those messages are required to be inside a block, because without braces only the first would belong to the `else` clause, and the second would print unconditionally regardless of whether the guess was right or wrong.

This distinction highlights why omitting necessary braces is dangerous. The following example looks as though `delta` is reset only when `depth` falls below the threshold, but it is not:

```
if (depth > 36.238)
   delta = 100;
else
   System.out.println ("WARNING: Delta is being reset to ZERO");
```

```
    delta = 0; // not part of the else clause!
```

The indentation implies that `delta = 0` is part of the `else` branch, but without enclosing braces it is not. It executes unconditionally every time, which is almost certainly not the programmer's intention. This is a clear illustration of why indentation alone cannot substitute for correct use of block syntax.

A block statement is syntactically valid anywhere a single statement is expected. Both the `if` branch and the `else` branch can independently be blocks, or both can be blocks simultaneously:

```
if (boxes != warehouse.getCount())

{

    System.out.println ("Inventory and warehouse do NOT match.");

    System.out.println ("Beginning inventory process again!");

    boxes = 0;

}

else

{

    System.out.println ("Inventory and warehouse MATCH.");

    warehouse.ship();

}
```

Here, the inventory count is compared against a value retrieved from the `warehouse` object. A mismatch triggers two printed messages and resets the counter. A match triggers a different message and calls the `ship` method.

The Conditional Operator

Java also provides a conditional operator that bears a functional resemblance to a compact `if-else`. It is a ternary operator — one of the very few in Java — meaning it requires three operands. Its symbol is written `?:`, though the two characters are always separated from each other by the operands between them. The following is an example:

```
(total > MAX) ? total + 1 : total * 2;
```

The boolean condition appears before the `?`. The two possible result expressions appear after it, separated by `:`. If the condition is `true`, the expression to the left of `:` is returned; if

the condition is `false`, the expression to the right of `:` is returned. Since the operator produces a value, that value is typically assigned somewhere:

```
total = (total > MAX) ? total + 1 : total * 2;
```

This is functionally equivalent to:

```
if (total > MAX)
   total = total + 1;
else
   total = total * 2;
```

Here is another practical example — initializing a variable to the larger of two values:

```
int larger = (num1 > num2) ? num1 : num2;
```

If `num1` exceeds `num2`, `larger` is initialized to `num1`; otherwise it is initialized to `num2`. Similarly, the following prints the smaller of the two values inline:

```
System.out.print ("Smaller: " + ((num1 < num2) ? num1 : num2));
```

The conditional operator is a useful tool in the right circumstances, but it is not a universal replacement for `if-else`. The `?:` operands must be expressions, not arbitrary statements, which limits where it can be applied. Even when it is technically applicable, the `if-else` formulation is often clearer. Readability should always take priority.

Nested `if` Statements

The statement controlled by an `if` can itself be another `if` statement, a pattern known as a nested `if`. This allows a second decision to be made contingent on the outcome of a first. The `MinOfThree` program in Listing 4.4 illustrates this pattern, using nested `if` statements to identify the smallest of three integers entered by the user.

Work through the logic of `MinOfThree` carefully, testing it mentally with different input arrangements — placing the minimum value in each of the three positions in turn — to confirm that it identifies the smallest value correctly in every case.

Nested `if` statements introduce an important syntactic ambiguity worth being aware of. When an `else` clause appears after a nested `if`, it might appear to belong to either the inner or the outer `if`. Consider:

```
if (code == 'R')
   if (height <= 20)
      System.out.println ("Situation Normal");
```

```
   else
      System.out.println ("Bravo!");
```

KEY CONCEPT

In a nested `if` statement, an `else` clause is matched to the closest unmatched `if`.

The indentation suggests the `else` belongs to the outer `if`, but Java's rule is unambiguous: an `else` clause is always paired with the nearest preceding unmatched `if`. In this example, that means the `else` belongs to the inner `if`, and "Bravo!" is printed when `code` equals `'R'` but `height` exceeds `20`.

If the intent is for "Bravo!" to print when `code` is not equal to `'R'`, braces must be used to override the default pairing and make the association explicit:

```
if (code == 'R')
{
   if (height <= 20)
      System.out.println ("Situation Normal");
}
else
   System.out.println ("Bravo!");
```

By wrapping the inner `if` in a block, the outer `if` is now paired with the `else`, which is the intended behavior. This example reinforces once more that accurate and consistent indentation, combined with careful use of braces, is essential for writing code whose behavior matches what the programmer intends.

```
// LISTING 3.1

//********************************************************************

// Age.java

//

// Demonstrates the use of an if statement.

//********************************************************************

import java.util.Scanner;

public class Age

{

   //-----------------------------------------------------------------
```

```java
    // Reads the user's age and prints comments accordingly.
    //-------------------------------------------------------------------
    public static void main(String[] args)
    {
        final int MINOR = 21;

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        System.out.println("You entered: " + age);

        if (age < MINOR)
            System.out.println("Youth is a wonderful thing, Enjoy.");

        System.out.println("Age is a state of mind.");
    }
}
```

O U T P U T

Enter your age: 43
You entered: 43
Age is a state of mind.

```java
// LISTING 3.2

//********************************************************************

// Wages.java

//

// Demonstrates the use of an if-else statement.

//********************************************************************

import java.text.NumberFormat;

import java.util.Scanner;


public class Wages

{

   //-----------------------------------------------------------------

   // Reads the number of hours worked and calculates wages.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       final double RATE = 8.25; // regular pay rate

       final int STANDARD = 40; // standard hours in a work week


       Scanner scanner = new Scanner(System.in);

       double pay = 0.0;


       System.out.print("Enter the number of hours worked: ");

       int hours = scanner.nextInt();

       System.out.println();


       // Pay overtime at "time and a half"
```

```
        if (hours > STANDARD)
            pay = STANDARD * RATE + (hours - STANDARD) * (RATE * 1.5);
        else
            pay = hours * RATE;

        NumberFormat fmt = NumberFormat.getCurrencyInstance();
        System.out.println("Gross earnings: " + fmt.format(pay));
    }
}
```

O U T P U T

Enter the number of hours worked: 46

Gross earnings: $404.25

```
// LISTING 3.3
//********************************************************************
// Guessing.java
//
// Demonstrates the use of a block statement in an if-else.
//********************************************************************
import java.util.*;

public class Guessing
{
   //-----------------------------------------------------------------
   // Plays a simple guessing game with the user.
   //-----------------------------------------------------------------
```

```
   public static void main(String[] args)

   {

       final int MAX = 10;

       int answer, guess;



       Scanner scanner = new Scanner(System.in);

       Random generator = new Random();



       answer = generator.nextInt(MAX) + 1;



       System.out.print("I'm thinking of a number between 1 and "

                       + MAX + ". Guess what it is: ");

       guess = scanner.nextInt();



       if (guess == answer)

           System.out.println("You got it! Good guessing!");

       else

       {

           System.out.println("That is not correct, sorry.");

           System.out.println("The number was " + answer);

       }

   }

}
```

O U T P U T

I'm thinking of a number between 1 and 10. Guess what it is: 4

That is not correct, sorry.

The number was 8

```java
// LISTING 3.4

//********************************************************************

// MinOfThree.java

//

// Demonstrates the use of nested if statements.

//********************************************************************

import java.util.Scanner;


public class MinOfThree

{

   //-----------------------------------------------------------------

   // Reads three integers from the user and determines the smallest

   // value.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       int num1, num2, num3, min = 0;


       Scanner scan = new Scanner(System.in);


       System.out.println("Enter three integers: ");

       num1 = scan.nextInt();

       num2 = scan.nextInt();

       num3 = scan.nextInt();
```

```
        if (num1 < num2)
            if (num1 < num3)
                min = num1;
            else
                min = num3;
        else
            if (num2 < num3)
                min = num2;
            else
                min = num3;



        System.out.println("Minimum value: " + min);
    }
}
```

O U T P U T

Enter three integers:

43 26 69

Minimum value: 26

4.3 Comparing Data

When evaluating boolean expressions that involve comparisons, it is essential to understand certain subtleties that arise depending on the type of data being compared. Let's examine several situations that warrant special attention.

Comparing Floats

A subtle but important complication arises when comparing floating point values. Two floating point numbers are considered equal by the `==` operator only when every binary digit of their underlying internal representations is identical. When the values being compared are the products of computation, they may differ by minute amounts even when conceptually they should be the same, making exact equality unlikely even if the values are practically indistinguishable for the purpose at hand. For this reason, the equality operator (`==`) should almost never be used when comparing floating point values.

A more reliable approach is to compute the absolute difference between the two values and compare that difference against some acceptable tolerance threshold. For instance, we might define a tolerance of `0.00001`. If the two values are close enough that their difference falls below this threshold, we treat them as equal. This comparison between two floating point values `f1` and `f2` could be written as:

```
if (Math.abs(f1 - f2) < TOLERANCE)
   System.out.println ("Essentially equal.");
```

The appropriate value for the `TOLERANCE` constant depends on the precision requirements of the specific problem being solved.

Comparing Characters

We have an intuitive understanding of what it means for one number to be less than another, but what does it mean for one character to be less than another? As we covered in Chapter 2, Java's character type is based on the Unicode character set, which imposes a well-defined numeric ordering on all representable characters. Because the character `'a'` appears before `'b'` in that ordering, we can meaningfully say that `'a'` is less than `'b'`.

KEY CONCEPT
The relative order of characters in Java is defined by the Unicode character set.

The equality and relational operators can be applied directly to character data. For example, given two character variables `ch1` and `ch2`, their relative positions in the Unicode ordering can be determined as follows:

```
if (ch1 > ch2)
   System.out.println (ch1 + " is greater than " + ch2);
else
   System.out.println (ch1 + " is NOT greater than " + ch2);
```

The Unicode character set is organized such that all lowercase alphabetic characters (`'a'` through `'z'`) occupy a contiguous, alphabetically ordered block. The same is true of the uppercase alphabetic characters (`'A'` through `'Z'`) and the digit characters (`'0'` through

'9'). In terms of their Unicode values, the digit characters come first, followed by the uppercase letters, followed by the lowercase letters, with various other characters interspersed before, after, and between these groups. Appendix C contains a detailed chart of the full character ordering.

Comparing Objects

The Unicode relationships among characters provide a natural and consistent foundation for sorting characters and, by extension, strings of characters. Given a list of names, for example, we can arrange them alphabetically by exploiting the inherent ordering of the characters that compose them.

KEY CONCEPT
The `compareTo` method can be used to determine the relative order of strings.

However, the equality and relational operators must not be used to compare `String` objects. The `String` class provides an `equals` method that returns a `boolean` value: `true` if the two strings being compared consist of exactly the same sequence of characters, and `false` otherwise. For example:

```
if (name1.equals(name2))
   System.out.println ("The names are the same.");
else
   System.out.println ("The names are not the same.");
```

Assuming `name1` and `name2` are `String` objects, this condition checks whether their character content is identical. Since both are instances of the `String` class, either object can be the one through which `equals` is invoked — writing `name2.equals(name1)` would produce the same outcome.

It is syntactically legal to write `(name1 == name2)`, but this does not test whether the two strings contain the same characters. For any object type, the `==` operator checks whether both reference variables point to the exact same object in memory — that is, whether they are aliases of one another sharing the same address. That is an entirely different question from whether two distinct `String` objects happen to hold the same text.

There is a related subtlety worth understanding. Java reuses `String` objects for identical string literals when possible. A string literal like `"Howdy"` is syntactic shorthand for creating a `String` object, and if the same literal appears multiple times within a method, Java creates only one object to represent it. As a result, both conditions in the following code will evaluate to `true`:

```
String str = "software";
if (str == "software")
```

```
    System.out.println ("References are the same");
if (str.equals("software"))
    System.out.println ("Characters are the same");
```

The first time the string literal `"software"` is encountered, a `String` object is created and `str` is set to its memory address. Every subsequent use of that same literal within the method refers back to the same object. This behavior, however, is an implementation detail of how Java handles literals and should not be relied upon for general string comparison.

To determine the relative ordering of two strings rather than just their equality, use the `compareTo` method of the `String` class. Unlike `equals`, which returns only a `boolean`, `compareTo` returns an `int` that encodes the nature of the ordering relationship. A negative return value indicates that the object on which the method is called lexicographically precedes the string passed as a parameter. A return value of zero means the two strings are character-for-character identical. A positive return value indicates that the calling object follows the parameter string in the ordering. For example:

```
int result = name1.compareTo(name2);
if (result < 0)
    System.out.println (name1 + " comes before " + name2);
else
    if (result == 0)
        System.out.println ("The names are equal.");
    else
        System.out.println (name1 + " follows " + name2);
```

It bears repeating that string comparisons are grounded in the Unicode character set (see Appendix C), following what is known as lexicographic ordering. When all characters in the strings being compared are of the same case, lexicographic order aligns with standard alphabetical order. However, when case is mixed — for instance, comparing `"able"` with `"Baker"` — `compareTo` will rank `"Baker"` first, because all uppercase letters precede all lowercase letters in the Unicode ordering. Additionally, when one string is a prefix of another — such as comparing `"horse"` with `"horsefly"` — `compareTo` considers the shorter string to come first.

4.4 The `switch` Statement

Java provides a second conditional construct called the `switch` statement, which directs program execution along one of several distinct paths based on the value of a single expression. The `break` statement is also discussed in this section because it is almost invariably used in conjunction with `switch`.

KEY CONCEPT

A `break` statement is usually used at the end of each case alternative of a `switch` statement.

The `switch` statement begins by evaluating an expression to produce a value, then scans its list of cases to find one whose label matches that value. When a matching case is found, execution continues with the statements associated with that case. Consider the following example:

```
switch (idChar)
{
   case 'A':
      aCount = aCount + 1;
      break;
   case 'B':
      bCount = bCount + 1;
      break;
   case 'C':
      cCount = cCount + 1;
      break;
   default:
      System.out.println ("Error in Identification Character.");
}
```

The expression is evaluated first — in this case it is simply a `char` variable. Control then transfers to the statement block of the first case whose label matches the evaluated value. If `idChar` holds `'A'`, then `aCount` is incremented. If it holds `'B'`, the `'A'` case is bypassed and execution resumes at the `'B'` case, incrementing `bCount`.

If none of the case labels match the evaluated value, execution falls through to the optional `default` case, introduced by the reserved word `default`. If no `default` case is present and no labels match, none of the statements inside the `switch` are executed and control moves to whatever follows the entire `switch` block. Including a `default` case is generally good practice, even in situations where you do not expect it to be reached.

The `break` statement causes execution to jump immediately to the statement following the closing brace of the `switch` block. Each case almost always ends with a `break` for this reason. Without it, execution would fall through into the next case regardless of its label. For example, if the `break` at the end of the `'A'` case were omitted, both `aCount` and `bCount` would be incremented whenever `idChar` contains `'A'` — almost certainly not the intended behavior. Occasionally the fall-through characteristic is intentionally exploited, but such situations are relatively rare.

The expression evaluated at the start of a `switch` statement must be of type `char`, `byte`, `short`, or `int`. It cannot be a `boolean`, a floating point value, or a `String`. Additionally,

every case label must be a constant — variables and expressions are not permitted as case labels.

Note that any `switch` statement can always be rewritten as a chain of nested `if` statements. However, deeply nested `if` constructs quickly become difficult to read and prone to errors in both writing and debugging. That said, because a `switch` tests only for equality, there are situations where nested `if` statements are unavoidable — specifically, when the condition involves a relational comparison other than equality. The right choice depends on the specifics of the situation.

The `GradeReport` program in Listing 3.5 illustrates how a `switch` can be applied creatively. It prompts the user for a numeric grade and prints an appropriate descriptive comment.

In `GradeReport`, the grade is first divided by `10` using integer division, collapsing it into an integer between `0` and `10` (assuming a valid grade is entered). This reduced value is then used as the `switch` expression, enabling the program to print tailored messages for each grade band at or above `60`, while the `default` case handles all failing grades.

```
// LISTING 3.5

//********************************************************************

// GradeReport.java

//

// Demonstrates the use of a switch statement.

//********************************************************************

import java.util.Scanner;


public class GradeReport

{

   //-----------------------------------------------------------------

   // Reads a grade from the user and prints comments accordingly.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {
```

```
        int grade, category;


        Scanner scan = new Scanner(System.in);


        System.out.print("Enter a numeric grade (0 to 100): ");
        grade = scan.nextInt();
        category = grade / 10;


        System.out.print("That grade is ");


        switch (category)
        {
            case 10:
                System.out.println("a perfect score. Well done.");
                break;
            case 9:
                System.out.println("well above average. Excellent.");
                break;
            case 8:
                System.out.println("above average. Nice job.");
                break;
            case 7:
                System.out.println("average.");
                break;
            case 6:
                System.out.print("below average. Please see the ");
                System.out.println("instructor for assistance.");
```

```
                break;

            default:

                System.out.println("not passing.");

        }

    }

}
```

O U T P U T

Enter a numeric grade (0 to 100): 87
That grade is above average. Nice job.

4.5 The `while` Statement

As introduced at the start of this chapter, a repetition statement — commonly called a loop — enables a statement to be executed multiple times. The `while` statement evaluates a boolean condition in the same manner as an `if` statement, and if that condition is `true`, it executes the body of the loop. The critical distinction from `if` is what happens next: after the body executes, the condition is evaluated again. If it remains `true`, the body runs again. This cycle continues, re-evaluating the condition and re-executing the body, until the condition becomes `false`, at which point execution continues with whatever statement follows the loop.

KEY CONCEPT
A `while` statement executes the same statement repeatedly until its condition becomes false.

The following loop prints the integers from `1` to `5`, incrementing a counter on each pass:

```
int count = 1;
while (count <= 5)
{
   System.out.println (count);
   count++;
}
```

The body of this loop is a block containing two statements, both of which are executed on every iteration.

The `Average` program in Listing 3.6 further demonstrates the `while` loop. It reads a series of integers from the user, accumulates their sum, and ultimately computes their average.

Since we cannot know in advance how many values the user intends to enter, the program needs a way to signal that input is complete. Here, the value `0` is designated as a sentinel value — a special input that indicates the end of the data stream rather than being a data value itself. The `while` loop continues accepting values until the user enters `0`. A sentinel must always lie outside the valid range of data values so it cannot be mistaken for legitimate input.

The variable `sum` maintains a running total: it begins at `0` and each new value is added to it as it is read. A separate counter tracks how many valid values have been entered, enabling the final average to be computed correctly by dividing by the right number. The sentinel value itself is not counted. The program also guards against the edge case in which the user immediately enters `0` without providing any other values, using an `if` statement to avoid a division-by-zero error.

The `WinPercentage` program in Listing 3.7 provides another application of the `while` loop, this time for input validation — ensuring that the value the user enters falls within an acceptable range before proceeding.

The loop keeps prompting the user until the number of games won is within the valid bounds: not less than `0` and not greater than the total number of games played. If the user's very first entry is valid, the loop body never executes at all. This pattern — using a loop to reject out-of-range input and re-prompt until valid data is received — is a standard technique for building robust programs. Striving for robustness means anticipating potential failure conditions, such as invalid input or division by zero, and handling them cleanly rather than allowing the program to crash or produce meaningless results.

Infinite Loops

KEY CONCEPT
We must design our programs carefully to avoid infinite loops.

It is the programmer's responsibility to ensure that a loop's controlling condition will eventually evaluate to `false`. If it never does, the loop body will execute without end — a situation known as an infinite loop, and one of the most common programming mistakes.

The following is a straightforward example of an infinite loop:

```
int count = 1;
while (count <= 25) // Warning: this is an infinite loop!
{
   System.out.println (count);
   count = count - 1;
}
```

If this loop is executed, be ready to interrupt it. On most systems, pressing Ctrl+C will terminate a running program.

The mistake here is that `count` starts at `1` and is decremented with each iteration. The condition checks whether `count` is less than or equal to `25`, but since `count` only gets smaller, the condition is always satisfied. The value will eventually underflow, but the logical error is obvious: the loop variable is moving in the wrong direction.

Here is another variety of infinite loop:

```
int count = 1;
while (count != 50) // infinite loop
   count += 2;
```

The variable begins at `1` and is incremented by `2` each time. The condition checks whether `count` has reached `50`, but since `count` moves through only odd values — `1`, `3`, `5`, ..., `49`, `51`, `53`, ... — it skips over `50` entirely and continues growing forever.

A third and subtler example:

```
double num = 1.0;
while (num != 0.0) // infinite loop
   num = num - 0.1;
```

Intuitively it appears that `num` will eventually reach exactly `0.0`, but in practice it almost certainly will not. Due to the way floating point values are encoded in binary, each subtraction of `0.1` introduces a tiny rounding error, and `num` will never hold a value that is precisely `0.0`. This is the same underlying issue we encountered earlier when discussing the dangers of using `==` to compare floating point values.

Nested Loops

The body of one loop can itself contain another loop — a structure called a nested loop. For every single iteration of the outer loop, the inner loop runs from start to finish. Consider the following code fragment and ask yourself: how many times is the string `"Here again"` printed?

```
int count1 = 1, count2;
while (count1 <= 10)
{
   count2 = 1;
   while (count2 <= 50)
   {
      System.out.println ("Here again");
```

```
        count2++;
    }
    count1++;
}
```

The outer loop executes `10` times as `count1` steps from `1` to `10`. For each of those `10` iterations, the inner loop runs `50` times as `count2` steps from `1` to `50`. The `println` statement therefore executes `10 × 50 = 500` times in total.

Small variations in the loop conditions or variable initializations can significantly change the result. If the outer loop condition were changed from `count1 <= 10` to `count1 < 10`, the outer loop would run only `9` times, reducing the total to `450` executions. If `count2` were initialized to `10` instead of `1` before the inner loop, the inner loop would execute only `40` times per outer iteration, yielding a total of `400`.

The `PalindromeTester` program in Listing 3.8 provides a practical application of nested loops. A palindrome is a sequence of characters that reads identically whether traversed left to right or right to left. Examples include:

- radar
- drab bard
- ab cde xxxx edc ba
- kayak
- deified
- able was I ere I saw elba

Note that palindromes may have either an even or an odd number of characters. The `PalindromeTester` program determines whether a string entered by the user is a palindrome, and allows the user to test as many strings as they wish.

The outer loop controls repetition across multiple test strings, continuing as long as the user wishes to keep testing. The inner loop performs the actual palindrome check, comparing characters from opposite ends of the string and advancing the two indices toward the center on each iteration.

The variables `left` and `right` hold the indices of the characters currently being compared. They start at the two ends of the string and move inward. The inner loop continues as long as the characters at both positions match and the indices have not yet crossed. If the loop exits because the characters no longer match, the string is not a palindrome. If it exits because `left` has reached or passed `right`, every pair was a match and the string is confirmed to be a palindrome.

It is worth noting that the following phrases would not be recognized as palindromes by the current implementation:

- A man, a plan, a canal, Panama.
- Dennis and Edna sinned.

- Rise to vote, sir.
- Doom an evil deed, liven a mood.
- Go hang a salami; I'm a lasagna hog.

These fail the current test because the program does not account for spaces, punctuation, or differences in letter case. Modifications that strip or ignore these elements are left as a programming project at the end of the chapter.

Other Loop Controls

We have already seen the `break` statement used to exit from individual cases within a `switch` statement. The `break` statement can also appear inside the body of any loop, where it causes the loop to terminate immediately and transfers control to the statement following the loop — analogous to its effect inside a `switch`.

That said, using `break` inside a loop is generally considered poor practice. Any loop that relies on a `break` can always be rewritten without one, and because `break` causes execution to jump abruptly from one location to another, it makes the flow of control harder to follow. Its use in `switch` statements is accepted because there is no clean equivalent without it; inside loops, that justification does not apply. It can and should be avoided.

The `continue` statement works similarly but with a different outcome. Rather than terminating the loop, `continue` abandons the remainder of the current iteration and causes the loop condition to be evaluated again, resuming normal loop execution if the condition is still `true`. Like `break`, the `continue` statement can always be eliminated from a loop, and for the same reasons of clarity and readability, it should be.

4.6 Iterators

An iterator is an object that provides methods for stepping through a collection of items one at a time, interacting with each item as needed. The goal might be to calculate membership dues for everyone in a club, break a URL into its constituent parts, or process a batch of returned library books. In each case, an iterator provides a consistent and straightforward mechanism for working through a group of items in a systematic way. Because this kind of processing is inherently repetitive, it connects naturally to our discussion of loops.

From a technical standpoint, iterator objects in Java are defined using the `Iterator` interface. For now, it is simply useful to know that such objects exist and that they offer a clean way to handle the sequential processing of a collection.

Every iterator object exposes a method called `hasNext` that returns a boolean value indicating whether at least one more item remains to be processed. This makes `hasNext`

well suited for use as the condition of a loop. Iterators also provide a method called `next` that retrieves the subsequent item from the collection.

KEY CONCEPT
An iterator is an object that helps you process a group of related items.

Several classes in the Java standard library define iterator objects. One of them is the `Scanner` class, which we have used repeatedly in earlier examples to read input from the user. The `hasNext` method of `Scanner` returns `true` as long as another input token is available to be read. And as we have already seen, the `next` method retrieves that next token as a string.

The `Scanner` class also provides type-specific variants of `hasNext` — such as `hasNextInt` and `hasNextDouble` — that allow us to check whether the next token is of a particular type before attempting to read it. Corresponding type-specific `next` variants like `nextInt` and `nextDouble` then retrieve the value in the appropriate form.

When reading interactively from the standard input stream, the `hasNext` method will block and wait for the user to type something before returning `true`. In other words, the keyboard is always treated as a source with more data potentially incoming — it simply has not been typed yet. This is precisely why earlier examples relied on sentinel values to signal the end of interactive input rather than on `hasNext` alone.

Where iterators become especially powerful, however, is when a `Scanner` is directed at a source with a well-defined endpoint — such as a data file or a fixed string. Let's look at a concrete example.

Reading Text Files

Suppose we have an input file named `websites.inp` containing a list of web page addresses (Uniform Resource Locators, or URLs) that we want to process. The first several lines of that file look like this:

```
www.google.com
newsyllabus.com/about
java.sun.com/j2se/6.0
www.linux.org/info/gnu.html
technorati.com/search/java/
www.cs.vt.edu/undergraduates/honors_degree.html
```

The `URLDissector` program in Listing 3.9 reads each URL from this file and breaks it apart to display its individual path components. It uses multiple `Scanner` objects — one to read lines from the file and a second to parse the tokens within each URL string.

The program contains two nested `while` loops. The outer loop processes each line of the input file in turn; the inner loop processes each token within the current line.

The variable `fileScan` is configured as a scanner reading from the file `websites.inp`. Rather than passing `System.in` to the `Scanner` constructor as we have done in interactive examples, we instantiate a `File` object that represents the file on disk and pass that into the constructor. Once created, `fileScan` is ready to read from the file.

KEY CONCEPT

The delimiters used to separate tokens in a `Scanner` object can be explicitly set as needed.

If the file cannot be found or opened for any reason, creating the `File` object will throw an `IOException`. That is why the `main` method header includes a `throws IOException` clause. Exception handling is explored more thoroughly in Chapter 10.

The body of the outer loop executes as long as `fileScan.hasNext()` returns `true` — that is, as long as there is more content remaining in the file. Each iteration reads one line (one URL) from the file and prints it.

For each URL, a fresh `Scanner` object called `urlScan` is created with that URL string passed directly into the constructor. By default, a `Scanner` uses whitespace as its delimiter, which works fine for reading individual lines from the file. However, within a URL the meaningful separators are forward slashes, not spaces. A call to `useDelimiter` on the `urlScan` object sets the delimiter to `"/"` before the inner loop processes the URL's components. The inner `while` loop then prints each delimited token on its own line.

For more complex parsing requirements — for instance, if multiple alternate delimiter characters are needed, or if the input requires pattern matching — the `Scanner` class supports full regular expression patterns.

```
// LISTING 3.9

//*************************************************************************

// URLDissector.java

//

// Demonstrates the use of Scanner to read file input and parse it

// using alternative delimiters.

//*************************************************************************


import java.util.Scanner;
```

```java
import java.io.*;


public class URLDissector

{

   //-----------------------------------------------------------------

   // Reads urls from a file and prints their path components.

   //-----------------------------------------------------------------

   public static void main(String[] args) throws IOException

   {

       String url;

       Scanner fileScan, urlScan;


       fileScan = new Scanner(new File("websites.inp"));


       // read and process each line of the file

       while (fileScan.hasNext())

       {

           url = fileScan.nextLine();

           System.out.println("URL: " + url);


           urlScan = new Scanner(url);

           urlScan.useDelimiter("/");


           // Print each part of the url

           while (urlScan.hasNext())

               System.out.println(" " + urlScan.next());
```

```
            System.out.println();

        }

    }

}
```

O U T P U T

URL: www.google.com

www.google.com

URL: newsyllabus.com/about

newsyllabus.com

about

URL: java.sun.com/j2se/6.0

java.sun.com

j2se

6.0

URL: www.linux.org/info/gnu.html

www.linux.org

info

gnu.html

URL: technorati.com/search/java/

technorati.com

search

java

URL: www.cs.vt.edu/undergraduates/honors_degree.html

www.cs.vt.edu

undergraduates

honors_degree.html

4.7 The do Statement

The `do` statement is closely related to the `while` statement — it also repeats its body until a controlling condition becomes false. The key distinction lies in when that condition is checked. In a `while` loop, the condition is evaluated before the body executes, meaning the body may not run at all if the condition is initially false. In a `do` loop, the body executes first, and only then is the condition checked. Syntactically, the condition appears after the body to reflect this ordering. This guarantees that the loop body runs at least once, regardless of the initial state of the condition.

KEY CONCEPT
A `do` statement executes its loop body at least once.

The following code prints the integers from `1` to `5` using a `do` loop. Compare it with the equivalent `while` loop example presented earlier in the chapter:

```
int count = 0;

do

{

   count++;

   System.out.println (count);

}

while (count < 5);
```

A `do` loop begins with the reserved word `do`, followed by the loop body. The `while` clause containing the boolean condition appears at the very end and is followed by a semicolon. Readers encountering a line beginning with `while` must look at the surrounding context to

determine whether it is the start of a `while` loop or the closing clause of a `do` loop — a potential source of confusion that is worth being mindful of.

The `ReverseNumber` program in Listing 3.10 offers a practical illustration of the `do` loop. It reads a positive integer from the user and reverses its digits through arithmetic rather than string manipulation.

On each iteration of the `do` loop, the remainder operation isolates the rightmost digit of the current value, that digit is incorporated into the reversed number, and integer division strips it from the original. The loop terminates when no digits remain — that is, when the variable `number` reaches zero. Trace through the program manually with a few sample inputs to verify that the logic produces the correct reversal.

Whenever you know that the loop body should execute at least once before any condition is checked, the `do` statement is the appropriate choice. Like any loop, it must be written carefully to ensure the termination condition will eventually be reached and an infinite loop is avoided.

```
// LISTING 3.10

//********************************************************************

// ReverseNumber.java

//

// Demonstrates the use of a do loop.

//********************************************************************

import java.util.Scanner;


public class ReverseNumber

{

   //-----------------------------------------------------------------

   // Reverses the digits of an integer mathematically.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       int number, lastDigit, reverse = 0;
```

```
        Scanner scanner = new Scanner(System.in);


        System.out.print("Enter a positive integer: ");
        number = scanner.nextInt();


        do
        {
            lastDigit = number % 10;
            reverse = (reverse * 10) + lastDigit;
            number = number / 10;
        }
        while (number > 0);


        System.out.println("That number reserved is " + reverse);
    }
}
```

O U T P U T

Enter a positive integer: 2896

That number reversed is 6982

4.8 The `for` Statement

The `while` and `do` statements are well suited to situations where the number of iterations is not known until the loop is actually running. The `for` statement is a third loop construct that excels when the number of iterations is fixed or can be determined before the loop begins.

KEY CONCEPT

A `for` statement is usually used when a loop will be executed a set number of times.

The following code prints the integers from `1` to `5` using a `for` loop, producing the same output as the equivalent `while` and `do` examples shown earlier:

```
for (int count=1; count <= 5; count++)

   System.out.println (count);
```

The header of a `for` loop contains three distinct parts separated by semicolons. The first part, called the initialization, is executed exactly once before the loop begins. The second part is the boolean condition, which is evaluated before each potential execution of the loop body — just as in a `while` loop. If the condition is `true`, the body executes. After each execution of the body, the third part — called the increment — is executed. The initialization runs only once; the condition and increment are evaluated and executed repeatedly.

Reading a `for` loop can take some getting used to. The increment appears in the header, but it executes after the body — not before. The execution order does not follow a simple top-to-bottom reading of the code.

In this example, the initialization declares the variable `count` and assigns it an initial value. Declaring the loop control variable directly in the header is common practice when that variable is not needed outside the loop. A variable declared in this way is scoped to the loop itself — it cannot be referenced anywhere after the closing brace of the loop body.

The loop control variable is set up, checked, and updated entirely through the header. It may be read within the loop body, but it should not be modified there — that responsibility belongs to the actions defined in the header.

Despite being called the "increment" portion, the third part of the header is not restricted to incrementing. It can perform any computation. The following loop, for instance, counts downward from `100` to `1`:

```
for (int num = 100; num > 0; num--)

   System.out.println (num);
```

The `Multiples` program in Listing 4.11 demonstrates a `for` loop whose increment adds a user-supplied value on each pass rather than simply stepping by one.

In that program, the loop begins at the first multiple of the entered value and adds that value on each iteration, continuing until the running multiple exceeds the specified upper limit. A counter tracks how many values have been printed on the current line, and a new line is started whenever the count is evenly divisible by the `PER_LINE` constant.

The `Stars` program in Listing 4.12 shows nested `for` loops in action. The output is a right triangle composed of asterisk characters. The outer loop runs exactly `MAX_ROWS` times, with each iteration producing one row of the triangle. The inner loop runs a number of times equal to the current row number, printing one asterisk per iteration. As the outer loop advances,

each successive row contains one more asterisk than the previous, forming the triangular shape.

Iterators and `for` Loops

In section 4.6 we established that some objects act as iterators, providing `hasNext` and `next` methods for processing a collection item by item. When an object implements the `Iterable` interface, Java offers a streamlined variant of the `for` loop that simplifies this kind of processing. For example, if `bookList` is an `Iterable` object containing `Book` objects, each element can be processed as follows:

```
for (Book myBook : bookList)
    System.out.println (myBook);
```

This construct is known as a for-each statement. It advances through the collection automatically, assigning each successive element to `myBook` in turn. It is functionally equivalent to:

```
Book myBook;
while (bookList.hasNext())
{
    myBook = bookList.next();
    System.out.println (myBook);
}
```

It is worth noting that the `Scanner` class is an `Iterator` but not `Iterable`. It has `hasNext` and `next` methods, but it cannot be used with the for-each loop syntax. Arrays, which are introduced in Chapter 7, are `Iterable` and can be used with the for-each loop. We will apply this construct in various appropriate contexts throughout the remainder of the book.

Comparing Loops

The three loop constructs — `while`, `do`, and `for` — are functionally interchangeable. Any loop written with one of them can always be rewritten using either of the others. The choice of which to use comes down to the characteristics of the situation at hand.

The most fundamental distinction between `while` and `do` is the timing of condition evaluation. If you know the loop body must execute at least once, the `do` loop is generally

the cleaner choice. The `while` loop, by contrast, may not execute its body at all if the condition is false from the outset. We therefore say that a `while` loop body runs zero or more times, whereas a `do` loop body always runs one or more times.

A `for` loop resembles a `while` loop in that its condition is evaluated before each execution of the body. The `for` loop's advantage lies in its header, which consolidates the initialization, condition, and update logic in a single, clearly visible location. This makes it the natural choice whenever the number of iterations is known or easily computable in advance, as it keeps the loop control logic concise and easy to scan.

Summary of Key Concepts

■ Conditionals and loops allow us to control the flow of execution through a method.
■ An `if` statement allows a program to choose whether to execute a particular statement.
■ A loop allows a program to execute a statement multiple times.
■ Logical operators are often used to construct sophisticated conditions.
■ Proper indentation is important for human readability; it shows the relationship between one statement and another.
■ An `if-else` statement allows a program to do one thing if a condition is true and another thing if the condition is false.
■ In a nested `if` statement, an `else` clause is matched to the closest unmatched `if`.
■ The relative order of characters in Java is defined by the Unicode character set.
■ The `compareTo` method can be used to determine the relative order of strings.
■ A `break` statement is usually used at the end of each case alternative of a `switch` statement.
■ A `while` statement executes the same statement repeatedly until its condition becomes false.
■ We must design our programs carefully to avoid infinite loops.
■ An iterator is an object that helps you process a group of related items.
■ The delimiters used to separate tokens in a `Scanner` object can be explicitly set as needed.
■ A `do` statement executes its loop body at least once.
■ A `for` statement is usually used when a loop will be executed a set number of times.